

Pontificia Universidad Católica del Ecuador Sede Manabí

Carrera de Software

María Emilia Navarrete Ávila

Prácticas en Gestión de la Información

Conexión a Base de datos desde Java usando JDBC

Ing. José Naranjo Villota

Grupo 1

06 de Julio de 2026

Índice

1. Definición y propósito	3
1.1. ¿Qué es JDBC?	3
1.2. Problema que resuelve	3
1.3. Importancia en el desarrollo de software	3
1.4. ¿Para qué sirve conectar Java con MySQL?	3
2. Componentes principales de JDBC	4
2.1. DriverManager	4
2.2. Conexión	4
2.3. Statement	5
2.4. ResultSet	5
3. Relación entre los componentes	6
3.1. Flujo de trabajo	7
4. Ventajas y desventajas	8
4.1. Ventajas	8
4.2. Desventajas	8
4.3. Comparativa con alternativas	8
5. Casos de uso	9
5.1. Ejemplos del mundo real	9
5.2. Cuando usarlo y cuando no	9
6. Mejores prácticas	10
6.1. Consejos de rendimiento y seguridad	10
7. Preparación del entorno de base de datos	10
7.1. Definición del esquema	10
8. Ejemplo práctico	11
9. Conclusiones	14

1. Definición y propósito

1.1. ¿Qué es JDBC?

JDBC (Java Database Connectivity) es la API oficial de Java que permite conectar una aplicación con una base de datos relacional como MySQL, PostgreSQL, Oracle o SQL Server. Esta API proporciona un conjunto de clases e interfaces que facilitan la comunicación entre una aplicación desarrollada en Java y el sistema gestor de bases de datos.

En otras palabras, JDBC actúa como un puente que permite que una aplicación Java pueda enviar instrucciones SQL a una base de datos y recuperar la información almacenada para ser utilizada dentro del programa.

1.2. Problema que resuelve

Las aplicaciones modernas necesitan almacenar información de forma permanente, como usuarios, productos, ventas, pedidos o clientes. Sin JDBC, una aplicación Java no podría comunicarse directamente con una base de datos.

JDBC resuelve este problema proporcionando una forma estandarizada de establecer conexiones, ejecutar consultas SQL, modificar datos y recuperar resultados sin importar el motor de base de datos utilizado.

1.3. Importancia en el desarrollo de software

JDBC constituye la base del acceso a datos en Java y es ampliamente utilizado en aplicaciones empresariales, sistemas web, aplicaciones de escritorio y servicios backend. Además, es el fundamento sobre el cual funcionan tecnologías más avanzadas como Hibernate o JPA.

1.4. ¿Para qué sirve conectar Java con MySQL?

Conectar Java con MySQL permite que una aplicación pueda almacenar y consultar información de forma permanente.

Gracias a JDBC una aplicación puede:

- Establecer una conexión con MySQL.
- Ejecutar consultas SQL.
- Insertar registros.
- Modificar información.

- Eliminar datos.
- Consultar información almacenada.
- Procesar los resultados obtenidos.

Por ejemplo, en un sistema de ventas, Java gestiona la lógica del negocio mientras que MySQL almacena toda la información de clientes, productos y facturas.

2. Componentes principales de JDBC

2.1. DriverManager

Es la clase responsable de administrar los controladores (drivers) de las bases de datos y establecer la conexión entre Java y MySQL.

Su método principal es:

```
DriverManager.getConnection(URL, USUARIO, CONTRASEÑA);
```

Función

- Localiza el driver JDBC adecuado.
- Establece la conexión con la base de datos.
- Devuelve un objeto de tipo Connection.

Ejemplo:

```
Connection conexion = DriverManager.getConnection(
"jdbc:mysql://localhost:3306/actividad_jdbc",
"root",
"1234");
```

2.2. Conexión

Representa la conexión activa entre la aplicación Java y la base de datos.

A través de este objeto se pueden:

- Crear consultas SQL.
- Administrar transacciones.

- Confirmar cambios.
- Cerrar la conexión.

Ejemplo:

```
Connection conexion =  
DriverManager.getConnection(URL, USUARIO, CONTRASEÑA);
```

2.3. Statement

Es el objeto encargado de enviar instrucciones SQL hacia la base de datos.

Se obtiene mediante:

```
Statement statement =  
conexion.createStatement();
```

Permite ejecutar:

Consultas:

```
statement.executeQuery();
```

Actualizaciones:

```
statement.executeUpdate();
```

2.4. ResultSet

Representa el conjunto de resultados obtenidos después de ejecutar una consulta SELECT.

Funciona como una tabla temporal que almacena los registros recuperados.

Ejemplo:

```
ResultSet resultado =  
statement.executeQuery("SELECT * FROM productos");
```

Los registros se recorren mediante:

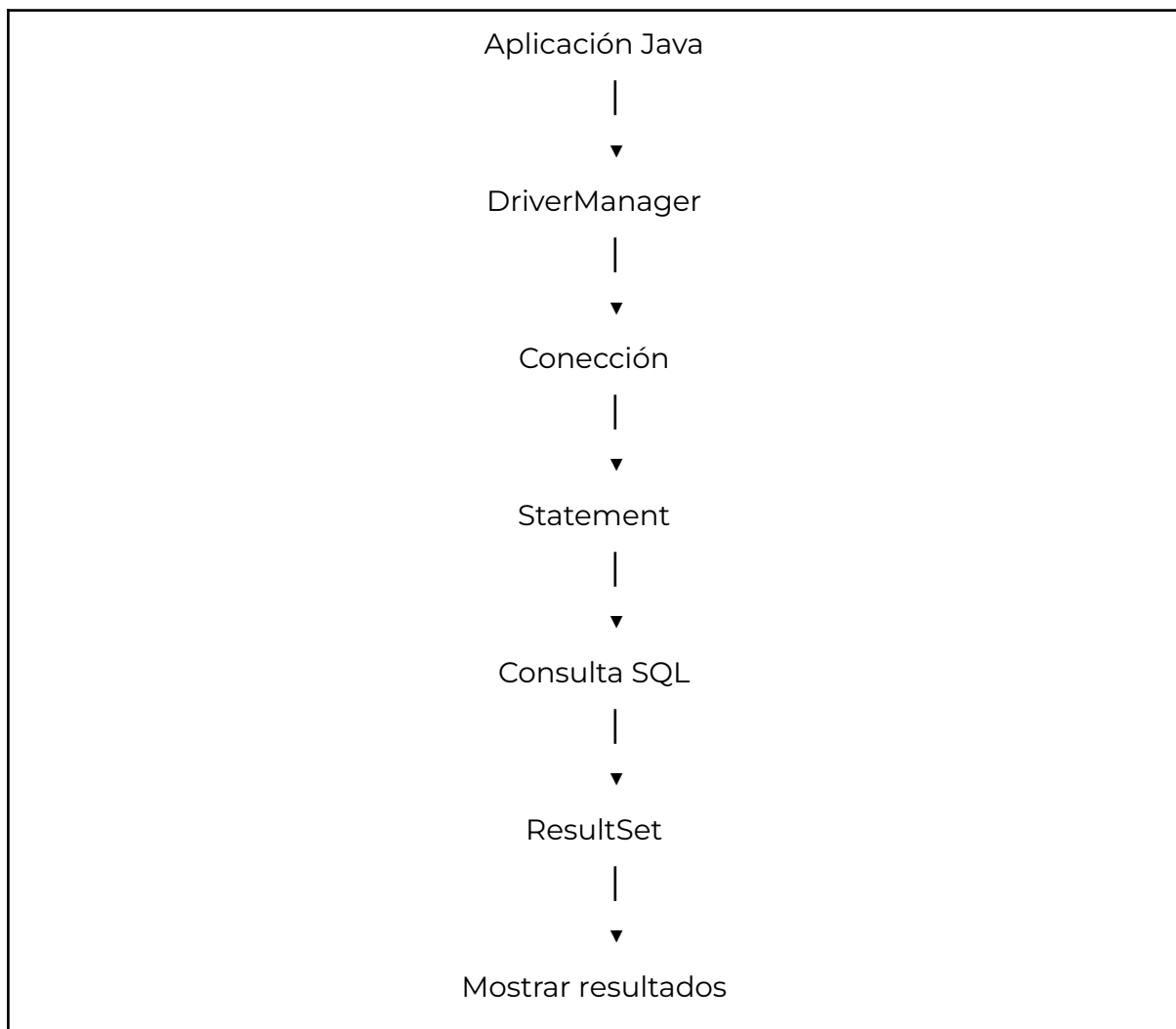
```
while(resultado.next())
```

Y cada columna puede obtenerse con métodos como:

```
resultado.getInt("id");  
resultado.getString("nombre");  
resultado.getDouble("precio");
```

3. Relación entre los componentes

El funcionamiento de JDBC sigue el siguiente flujo:



3.1. Flujo de trabajo

El proceso para conectar Java con MySQL mediante JDBC consta de seis pasos fundamentales:

Paso 1: Establecer la conexión utilizando DriverManager

- Se importan las clases del paquete java.sql (Connection, DriverManager, Statement, ResultSet). Desde JDBC 4.0 el driver se registra automáticamente mediante el mecanismo Service Provider Interface, siempre que la dependencia de MySQL esté declarada en el proyecto.

Paso 2: Crear el objeto Statement

- Se invoca DriverManager.getConnection(URL, USUARIO, CONTRASEÑA). DriverManager localiza el driver compatible con la URL y este abre la conexión real con el servidor. Es el paso más costoso del flujo, por lo que en aplicaciones con muchos usuarios se recomienda usar un pool de conexiones en lugar de abrir una nueva por cada operación.

Paso 3: Ejecutar la consulta SQL

- A partir de la Connection se invoca conexion.createStatement(), obteniendo el canal por el cual se envían las sentencias SQL. Es adecuado solo cuando el SQL no depende de datos del usuario; si los hubiera, se debe usar PreparedStatement para evitar inyección SQL.

Paso 4: Ejecutar y obtener resultados

- Se usa executeQuery() para sentencias SELECT, que devuelve un ResultSet, o executeUpdate() para INSERT, UPDATE y DELETE, que devuelve el número de filas afectadas y permite confirmar si la operación tuvo efecto.

Paso 5: Recorrer los resultados mediante ResultSet

- El ResultSet funciona como un cursor posicionado antes de la primera fila, por lo que se recorre con while (resultado.next()). Cada columna se obtiene por nombre (getString("nombre")) o posición, siendo el nombre más legible aunque levemente más costoso.

Paso 6: Cerrar automáticamente los recursos mediante try-with-resources

- Connection, Statement y ResultSet implementan AutoCloseable, por lo que Java los cierra en orden inverso al declararlos, incluso si ocurre una excepción. Esto evita dejar conexiones abiertas, causa común del error Too many connections en producción.

4. Ventajas y desventajas

4.1. Ventajas

- Es la API oficial de Java para acceso a datos.
- Compatible con múltiples motores de bases de datos.
- Permite ejecutar cualquier sentencia SQL.
- Alto rendimiento.
- Fácil integración con aplicaciones Java.
- Amplia documentación y soporte.

4.2. Desventajas

- Requiere escribir más código que un ORM.
- El desarrollador debe administrar manualmente las consultas SQL.
- Puede ser vulnerable a SQL Injection cuando se utiliza Statement con datos ingresados por el usuario.
- El mantenimiento puede ser más complejo en aplicaciones grandes.

4.3. Comparativa con alternativas

JDBC ofrece control total sobre las sentencias SQL y un alto rendimiento, ya que el desarrollador escribe exactamente la consulta que se ejecuta. En cambio, un ORM como Hibernate o JPA automatiza el mapeo entre los objetos Java y las tablas de la base de datos, reduciendo significativamente la cantidad de código repetitivo. Sin embargo, esta comodidad tiene un costo: el desarrollador pierde parte del control fino sobre las consultas generadas, y en operaciones muy específicas el rendimiento puede verse afectado si no se configura correctamente el ORM.

5. Casos de uso

JDBC se utiliza en aplicaciones Java que necesitan conectarse a una base de datos para almacenar, consultar, actualizar o eliminar información de forma segura y eficiente. Es una tecnología muy utilizada cuando los datos cambian constantemente y deben mantenerse organizados.

Se aplica en:

- Sistemas de inventario.
- Sistemas bancarios.
- Sistemas hospitalarios.
- Plataformas educativas.
- Sistemas de ventas.
- Comercio electrónico.
- Aplicaciones de escritorio.
- Aplicaciones web desarrolladas en Java.

5.1. Ejemplos del mundo real

- En un sistema bancario, JDBC permite consultar el saldo de una cuenta, registrar transferencias y actualizar los movimientos de los clientes.
- En una tienda en línea, se utiliza para guardar pedidos, actualizar el inventario y gestionar la información de los clientes.
- En una plataforma educativa, permite registrar estudiantes, almacenar calificaciones y consultar el historial académico.

5.2. Cuando usarlo y cuando no

JDBC es recomendable cuando se requiere control total sobre las sentencias SQL, un alto rendimiento, o cuando el proyecto es pequeño o mediano y no justifica la complejidad de un framework adicional. En cambio, no es la opción más práctica en proyectos grandes con muchas entidades relacionadas entre sí, ya que escribir y mantener manualmente todo el SQL se vuelve repetitivo y propenso a errores; en esos casos conviene usar un ORM como Hibernate o JPA, que automatiza gran parte de ese trabajo.

6. Mejores prácticas

Para desarrollar aplicaciones seguras y eficientes utilizando JDBC se recomienda:

- Utilizar **try-with-resources** para cerrar automáticamente Connection, Statement y ResultSet.
- Utilizar **PreparedStatement** cuando existan parámetros ingresados por el usuario.
- No almacenar contraseñas dentro del código fuente.
- Manejar correctamente las excepciones.
- Cerrar siempre las conexiones una vez finalizada la consulta.
- Validar los datos antes de enviarlos a la base de datos.

6.1. Consejos de rendimiento y seguridad

Para mejorar el rendimiento, es recomendable utilizar PreparedStatement, ya que permite reutilizar consultas y optimiza su ejecución. En aplicaciones con muchos usuarios, el uso de un pool de conexiones reduce el tiempo necesario para conectarse a la base de datos. Desde el punto de vista de la seguridad, las credenciales deben almacenarse en archivos de configuración o variables de entorno y no dentro del código fuente, además de mantener actualizados los controladores JDBC para corregir vulnerabilidades.

7. Preparación del entorno de base de datos

Antes de ejecutar el código fuente en Java, es necesario configurar la infraestructura de persistencia. Esto implica la creación de la base de datos, la estructura de la tabla y la carga de datos iniciales.

7.1. Definición del esquema

Para establecer la estructura de datos, se ejecuta un script SQL en la herramienta de administración de bases de datos (pgAdmin). Este script define la tabla pacientes utilizando el tipo de dato SERIAL para el identificador, lo que garantiza que cada nuevo registro obtenga un número de ID único y autoincrementable.

```
1  -- Crear la tabla pacientes
2  CREATE TABLE pacientes (
3      id SERIAL PRIMARY KEY,
4      nombre VARCHAR(50),
5      edad INT,
6      diagnostico VARCHAR(50),
7      habitacion VARCHAR(10),
8      estado VARCHAR(20)
9  );
10
11  -- Inserción de datos iniciales para validación
12  INSERT INTO pacientes (nombre, edad, diagnostico, habitacion, estado)
13  VALUES ('Carlos Molina', 45, 'Gripe', '101', 'Estable'),
14          ('Emilia Carrera', 32, 'Fractura', '204', 'En observación'),
15          ('Gabriela Loo', 60, 'Hipertensión', '302', 'Estable');
16
```

7.2. Configuración del Driver JDBC

Para que la aplicación Java pueda establecer comunicación con el servidor de base de datos PostgreSQL, es necesario integrar el controlador (driver) correspondiente en el proyecto:

- Integración: Se debe incluir el archivo .jar del controlador de PostgreSQL en la carpeta lib del proyecto.
- Configuración del Classpath: En el entorno de desarrollo (VS Code), se debe asegurar que el archivo .jar esté correctamente referenciado en el Classpath para que el motor de Java reconozca la clase org.postgresql.Driver durante la ejecución.

7.3. Validación de la conexión

Una vez creada la base de datos y configurado el entorno, se debe validar la conectividad antes de ejecutar la lógica principal del sistema:

- Credenciales: Verificar que el usuario y la contraseña configurados en el código coincidan con los permisos otorgados en pgAdmin.
- Prueba de consulta: Se puede ejecutar una consulta rápida en la consola de pgAdmin, para confirmar que los datos fueron persistidos correctamente y que la base de datos actividad_jdbc se encuentra en estado operativo.

```
16
17  SELECT * FROM pacientes;
18
```

8. Ejemplo práctico

```
/**
 * DEMOSTRACIÓN DE CONEXIÓN A BD DESDE JAVA USANDO JDBC
 * Objetivo: Conectar Java a MySQL con DriverManager, ejecutar consultas SQL
 con Statement y recorrer los resultados con ResultSet.
 * Simula un sistema hospitalario: lista los pacientes ingresados.
 * Autora: María Emilia Navarrete Ávila
 * Fecha: 15/07/2026
 */

package com.demo.jdbc;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;

public class conexionMySQLDemo {

    // URL de conexión: protocolo (jdbc:mysql) + host + puerto + nombre de la
base de datos
    private static final String URL = "jdbc:mysql://localhost:3306/actividad_jdbc";

    // Usuario de la base de datos
    private static final String USUARIO = "root";

    // Contraseña de la base de datos (en un proyecto real no debe escribirse la
contraseña, por seguridad)
    private static final String CONTRASEÑA = "1234";

    public static void main(String[] args) {

        try (
            // Abre la conexión real con la base de datos MySQL
```

```
Connection conexion = DriverManager.getConnection(URL, USUARIO,
CONTRASENA);

// Crea el objeto que se usará para enviar las sentencias SQL
Statement statement = conexion.createStatement()

// Al terminar el bloque try, Java cierra ambos automáticamente,
incluso si ocurre un error
){
    System.out.println("Conexión exitosa a MySQL");
    System.out.println();

    mostrarPacientes(statement);
    mostrarResumenHospitalario(statement);

} catch (Exception e) {
    // Aquí caen errores como credenciales incorrectas, servidor apagado, o
    que la tabla "pacientes" no exista
    System.out.println("Error en la conexión o consulta.");
    e.printStackTrace();
}
}

// Consulta todos los pacientes y los imprime en formato de tabla.
private static void mostrarPacientes(Statement statement) throws Exception {

    String sql = "SELECT id, nombre, edad, diagnostico, habitacion, estado "
        + "FROM pacientes ORDER BY id";

    // executeQuery() ejecuta el SELECT y devuelve un ResultSet
    try (ResultSet resultado = statement.executeQuery(sql)) {

        System.out.println("=== PACIENTES INGRESADOS ===");
        System.out.printf("%-4s %-15s %-5s %-15s %-6s %-10s%n",
            "ID", "NOMBRE", "EDAD", "DIAGNÓSTICO", "HAB.", "ESTADO");
```

```
System.out.println("-----");

    // next() mueve el cursor a la siguiente fila; devuelve false cuando ya no
    hay más
    while (resultado.next()) {

        int id = resultado.getInt("id");
        String nombre = resultado.getString("nombre");
        int edad = resultado.getInt("edad");
        String diagnostico = resultado.getString("diagnostico");
        String habitacion = resultado.getString("habitacion");
        String estado = resultado.getString("estado");

        System.out.printf("%-4d %-15s %-5d %-15s %-6s %-10s\n",
            id, nombre, edad, diagnostico, habitacion, estado);
    }
    System.out.println();
}

// Ejecuta una consulta de agregación (COUNT) para mostrar cuántos
pacientes hay en total.
private static void mostrarResumenHospitalario(Statement statement) throws
Exception {

    // COUNT(*) cuenta todas las filas de la tabla
    String sql = "SELECT COUNT(*) AS total_pacientes FROM pacientes";

    try (ResultSet resultado = statement.executeQuery(sql)) {
        System.out.println("=== RESUMEN HOSPITALARIO ===");

        // Esta consulta siempre devuelve una sola fila con los totales
        if (resultado.next()) {
            int totalPacientes = resultado.getInt("total_pacientes");
```

```
        System.out.println("Total de pacientes ingresados: " + totalPacientes);  
    }  
}  
}  
}
```

9. Conclusiones

- JDBC constituye la tecnología estándar para conectar aplicaciones Java con bases de datos relacionales.
- Los componentes **DriverManager**, **Connection**, **Statement** y **ResultSet** trabajan de manera integrada para establecer la comunicación entre Java y MySQL.
- El uso de buenas prácticas, como **try-with-resources** y el manejo adecuado de excepciones, permite desarrollar aplicaciones más seguras, eficientes y fáciles de mantener.
- JDBC representa la base para comprender tecnologías de persistencia más avanzadas, como **JPA** e **Hibernate**, utilizadas ampliamente en el desarrollo profesional de software.